

To write it, it took three months; to conceive it – three minutes; to collect the data in it – all my life. [F. Scott Fitzgerald](#)

Introduction

[sqldf](#) is an R package for running [SQL statements](#) on R data frames, optimized for convenience. The user simply specifies an SQL statement in R using data frame names in place of table names and a database with appropriate table layouts/schema is automatically created, the data frames are automatically loaded into the database, the specified SQL statement is performed, the result is read back into R and the database is deleted all automatically behind the scenes making the database's existence transparent to the user who only specifies the SQL statement. Surprisingly this can at times [be even faster than the](#) corresponding pure R calculation (although the purpose of the project is convenience and not speed). [This link](#) suggests that for aggregations over highly granular columns that sqldf is faster than another alternative tried. sqldf is free software published under the GNU General Public License that can be downloaded from [CRAN](#).

sqldf supports (1) the [SQLite](#) backend database (by default), (2) the [H2](#) java database, (3) the [PostgreSQL](#) database and (4) sqldf 0.4-0 onwards also supports [MySQL](#). SQLite, H2, MySQL and PostgreSQL are free software. SQLite and H2 are embedded serverless zero administration databases that are included right in the R driver packages, [RSQLite](#) and [RH2](#), so that there is no separate installation for either one. A number of [high profile projects](#) use SQLite. H2 is a java database which contains a large collection of SQL functions and supports Date and other data types. It is the most popular database package among [scala packages](#). PostgreSQL is a client/server database and unlike SQLite and H2 must be separately installed but it has a particularly powerful version of SQL, e.g. its [window functions](#), so the extra installation work can be worth it. sqldf supports the RPostgreSQL driver in R. Like PostgreSQL, MySQL is a client server database that must be installed independently so its not as easy to install as SQLite or H2 but its very popular and is widely used as the back end for web sites.

The information below mostly concerns the default SQLite database. The use of H2 with sqldf is discussed in [FAQ #10](#) which discusses differences between using sqldf with SQLite and H2 and also shows how to modify the code in the [Examples](#) section to use sqldf/H2 rather than sqldf/SQLite. There is some information on using PostgreSQL with sqldf in [FAQ #12](#) and an example in [Example 17. Lag](#). The unit tests provide examples that can work with all five data base drivers (covering four databases) supported by sqldf. They are run by loading whichever database is to be tested (SQLite is the default) and running: `demo("sqldf-unitTests")`

[Overview](#)

[Citing sqldf](#)

[For Those New to R](#)

[News](#)

[Troubleshooting](#)

- [Problem is that installer gives message that sqldf is not available](#)
- [Problem with no argument form of sqldf - sqldf\(\)](#)
- [Problem involving tcltk](#)

[FAQ](#)

- [1. How does sqldf handle classes and factors?](#)
- [2. Why does sqldf seem to mangle certain variable names?](#)
- [3. Why does sqldf\(“select var\(x\) from DF”\) not work?](#)
- [4. How does sqldf work with “Date” class variables?](#)
- [5. I get a message about the tcltk package being missing.](#)
- [6. Why are there problems when we use table names or column names that are the same except for case?](#)
- [7. Why are there messages about MySQL?](#)
- [8. Why am I having problems with update?](#)
- [9. How do I examine the layout that SQLite uses for a table? which tables are in the database? which databases are attached?](#)
- [10. What are some of the differences between using SQLite and H2 with sqldf?](#)
- [11. Why am I having difficulty reading a data file using SQLite and sqldf?](#)
- [12. How does one use sqldf with PostgreSQL?](#)

- [13. How does one deal with quoted fields in read.csv.sql ?](#)
- [14. How does one read files where numeric NAs are represented as missing empty fields?](#)
- [15. Why do certain calculations come out as integer rather than double?](#)
- [16. How can one read a file off the net or a csv file in a zip file?](#)

[Examples](#)

- [Example 1. Ordering and Limiting](#)
- [Example 2. Averaging and Grouping](#)
- [Example 3. Nested Select](#)
- [Example 4. Join](#)
- [Example 5. Insert Variables](#)
- [Example 6. File Input](#)
- [Example 7. Nested Select](#)
- [Example 8. Specifying File Format](#)
- [Example 9. Working with Databases](#)
- [Example 10. Persistent Connections](#)
- [Example 11. Between and Alternatives](#)
- [Example 12. Combine two files in permanent database](#)
- [Example 13. read.csv.sql and read.csv2.sql](#)
- [Example 14. Use of spatialite library functions](#)
- [Example 15. Use of RSQLite.extfuns library functions](#)
- [Example 16. Moving Average](#)
- [Example 17. Lag](#)
- [Example 18. MySQL Schema Information](#)

[Links](#)

Overview

[sqldf](#) is an R package for running [SQL statements](#) on R data frames, optimized for convenience. [sqldf](#) works with the [SQLite](#), [H2](#), [PostgreSQL](#) or [MySQL](#) databases. SQLite has the least prerequisites to install. H2 is just as easy if you have Java installed and also supports Date class and a few additional functions. PostgreSQL notably supports Windowing functions providing the SQL analogue of the R `ave` function. MySQL is a particularly popular database that drives many web sites.

More information can be found from within R by installing and loading the [sqldf](#) package and then entering [?sqldf](#) and [?read.csv.sql](#). A number of [examples](#) are on this page and more examples are accessible from within R in the examples section of the [?sqldf](#) help page.

As seen from this example which uses the built in BOD data frame:

```
library(sqldf)
sqldf("select * from BOD where Time > 4")
```

with [sqldf](#) the user is freed from having to do the following, all of which are automatically done:

- database setup
- writing the `create table` statement which defines each table
- importing and exporting to and from the database
- coercing of the returned columns to the appropriate class in common cases

It can be used for:

- learning SQL if you know R
- learning R if you know SQL
- as an alternate syntax for data frame manipulation, particularly for purposes of speeding these up, since [sqldf](#) with SQLite as the underlying database is often faster than performing the same manipulations in straight R
- reading portions of large files into R without reading the entire file (example 6b and example 13 below show two different ways and examples 6e, 6f below show how to read random portions of a file)

In the case of SQLite it consists of a thin layer over the [RSQLite DBI](#) interface to SQLite itself.

In the case of H2 it works on top of the [RH2 DBI](#) driver which in turn uses RJDBC and JDBC to interface to H2 itself.

In the case of PostgreSQL it works on top of the [RPostgreSQL DBI](#) driver.

There is also some untested code in sqldf for use with the [MySQL](#) database using the [RMySQL DBI](#) driver.

Citing sqldf

To get information on how to cite sqldf in papers, issue the R commands:

```
library(sqldf)
citation("sqldf")
```

For Those New to R

If you have not used R before and want to try sqldf with SQLite, [google for single letter R](#), download R, install it on Windows, Mac or UNIX/Linux and then start R and at R console enter this:

```
# installs everything you need to use sqldf with SQLite
# including SQLite itself
install.packages("sqldf")
# shows built in data frames
data()
# load sqldf into workspace
library(sqldf)
sqldf("select * from iris limit 5")
sqldf("select count(*) from iris")
sqldf("select Species, count(*) from iris group by Species")
# create a data frame
DF <- data.frame(a = 1:5, b = letters[1:5])
sqldf("select * from DF")
sqldf("select avg(a) mean, variance(a) var from DF") # see example 15
```

To try it with H2 rather than SQLite the process is similar. Ensure that you have the [java](#) runtime installed, install R as above and start R. From within R enter this ensuring that the version of RH2 that you have is RH2 0.1-2.6 or later:

```
# installs everything including H2
install.packages("sqldf", dep = TRUE)
# load RH2 driver and sqldf into workspace
library(RH2)
packageVersion("RH2") # should be version 0.1-2-6 or later
library(sqldf)
#
sqldf("select * from iris limit 5")
sqldf("select count(*) from iris")
sqldf("select Species, count(*) from iris group by Species")
DF <- data.frame(a = 1:5, b = letters[1:5])
sqldf("select * from DF")
sqldf("select avg(a) mean, var_samp(a) var from DF")
```

Troubleshooting

sqldf has been [extensively tested](#) with multiple architectures and database back ends but there are no guarantees.

Problem is that installer gives message that sqldf is not available

See <https://stackoverflow.com/questions/27772756/sqldf-doesnt-install-on-ubuntu-14-04>

Problem with no argument form of sqldf - sqldf()

The no argument form, i.e. `sqldf()` is used for opening and closing a connection so that intermediate `sqldf` statements can all use the same connection. If you have forgotten whether the last `sqldf()` opened or closed the connection this code will close it if it is open and otherwise do nothing:

```
# close an old connection if it exists
if (!is.null(getOption("sqldf.connection"))) sqldf()
```

Thanks to Chris Davis <https://groups.google.com/d/msg/sqldf/-YAvajnlRrY/7nF8tpBnrcAJ> for pointing this out.

Problem involving tcltk

The most common problem is that the `tcltk` package and `tcl/tk` itself are missing. Historically these were bundled with the Windows version of R so Windows users should not experience any problems on this account. Since R version 3.0.0 Mac versions of R also have the `tcltk` package and `Tcl/Tk` itself bundled so if you are having a problem on the Mac you may only need to upgrade to the latest version of R. If upgrading to the latest version of R does not help then using this line will usually allow it to work even without the `tcltk` package and `tcl/tk` itself:

```
options(gsubfn.engine = "R")
```

Running the above options line before using `sqldf`, e.g. put that options line in your `.Rprofile`, is all that is needed to get `sqldf` to work without the `tcltk` package and `tcl/tk` itself in most cases; however, this does have the downside that it will use the R engine which is slower. An alternative, is to rebuild R yourself as discussed here:

<https://permlink.gmane.org/gmane.comp.lang.r.fedora/235>

If the above does not resolve the problem then read the more detailed discussion below.

A related problem is that your R installation is flawed or incomplete in some way and the main way to fix that is to fix your installation of R. This will not only affect `sqldf` but also many other R packages so information on installing them can also help here. In particular [installation information for the Rcmdr package](#) may be useful since its likely that if you can install `Rcmdr` then you can also install `sqldf`.

- `sqldf` uses the `gsubfn` R package which normally uses the `tcltk` R package which in turn uses `tcl/tk` itself. The `tcltk` package is a core component of R so a complete distribution of R should have `tcltk` capability. For this to happen `tcl/tk` **must** be present at the time **R itself was built** (the build process automatically excludes `tcltk` capability if it does not sense that `tcl/tk` is present at the time R itself is built) but it is possible to run `gsubfn` and therefore also `sqldf` without `tcl/tk` present at the time `sqldf` runs (although it will run slower if you do this). There are three possibilities: (1) **tcltk capability absent**. If this command from within R `capabilities()[["tcltk"]]` is FALSE then your distribution of R was built without `tcltk` capability. In that case you **must** use a different distribution of R. All common distributions of R including the CRAN distribution for Windows and most distributions for Linux do have `tcltk` capability. Note that a given version of R may have been built with or without `tcltk` capability so simply checking which version of R you have won't tell you whether your distribution was built correctly. This situation mostly affects distributions of R built by the user or improperly built by others and then distributed. (2) **tcl/tk missing on system** (a) If your distribution of R was built with `tcltk` capability as described in the last point but you don't have `tcl/tk` itself on your system you can simply install `tcl/tk` yourself. In most cases this is actually quite easy to do – its typically a one line `apt-get` on Linux. There is information about installing `tcl/tk` near the end of [FAQ #5](#) or
b. if your distribution of R was built with `tcltk` capability as described in the first point but you don't have `tcl/tk` on your system and you don't want to bother to install it then issue the R command:

In that case `gsubfn` will use the slower R engine instead of the faster `tcltk` engine so you won't need `tcl/tk` installed on your system in the first place. Be sure you are using `gsubfn` 0.6-4 or later if you use this option since prior versions of `gsubfn` had a bug which could interfere with the use of this option. To check your version of `gsubfn`:

```
packageVersion("gsubfn")
```

- using an old version of R, `sqldf` or some other software. If that is the problem upgrade to the most recent versions [on CRAN](#). Also be sure you are using the latest versions of other packages used by `sqldf`. If you are getting `NAMESPACE` errors then this is likely the problem. You can find the current version of R [here](#) and then install `sqldf` from within R using `install.packages("sqldf")`. If you already have the current version of R and have installed the packages you want then you can update your installed packages to the current version by entering this in R: `update.packages()`. In most cases all the mirrors are up to date but if that should fail to update to the most recent packages on CRAN then try using a more up to date mirror.

- unexpected errors concerning H2, MySQL or PostgreSQL. sqldf automatically uses H2, MySQL or PostgreSQL if the R package RH2, RMySQL or RpgSQL is loaded, respectively. If none of them are loaded it uses sqlite. To force it to use sqlite even though one of those others is loaded (1) add the `drv = "SQLite"` argument to each sqldf call or (2) issue the R command:

in which case all sqldf calls will use sqlite. See [FAQ #7](#) for more info.

- message about tcltk being missing or other tcltk problem. This is really the same problem discussed in the first point above. Upgrade to sqldf 0.4-5 or later. If it still persists then set this option: `options(gsubfn.engine = "R")` which causes R code to be substituted for the tcl code or else just install the tcltk package. See [FAQ #5](#) for more info. If you installed the tcltk package and it still has problems then remove the tcltk package and try these steps again.
- error messages regarding a data frame that has a dot in its name. The dot is an SQL operator. Either quote the name appropriately or change the name of the data frame to one without a dot.
- as recommended in the [INSTALL](#) file its better to install sqldf using `install.packages("sqldf")` and **not** `install.packages("sqldf", dep = TRUE)` since the latter will try to pull in every R database driver package supported by sqldf which increases the likelihood of a problem with installation. Its unlikely that you need every database that sqldf supports so doing this is really asking for trouble. The recommended way does install sqlite automatically anyways and if you want any of the additional ones just install them separately.
- Mac users. According to <http://cran.us.r-project.org/bin/macosx/tools/> Tcl/Tk comes with R 3.0.0 and later but if you are using an earlier version of R look at [this link](#).

FAQ

1. How does sqldf handle classes and factors?

sqldf uses a heuristic to assign classes and factor levels to returned results. It checks each column name returned against the column names in the input data frames and if the output column name matches any input column name then it assigns the input class to the output. If two input data frames have the same column names then this automatic assignment is disabled if they differ in class. Also if `method = "raw"` then the automatic class assignment is disabled. This also extends to factor levels as well so that if an output column corresponds to an input column that is of class "factor" then the factor levels of the input column are assigned to the output column (again assuming that only one input column has the output column name). Also in the case of factors the levels of the output must appear among the levels of the input.

sqldf knows about Date, POSIXct and chron (dates, times) classes but not POSIXlt and other date and time classes.

Previously this section had an example of how the heuristic could go awry but improvements in the heuristic in sqldf 0.4-0 are such that that example now works as expected.

2. Why does sqldf seem to mangle certain variable names?

Starting with RSQLite 1.0.0 and sqldf 0.4-9 dots in column names are no longer translated to underscores.

If you are using an older version of these packages then note that since dot is an SQL operator the RSQLite driver package converts dots to underscores so that SQL statements can reference such columns unquoted.

Also note that certain names are SQL keywords. These can be found using this code:

```
.SQL92Keywords
```

Note that using such names can sometimes result in an error message such as:

```
Error in sqliteExecStatement(con, statement, bind.data) :  
  RS-DBI driver: (error in statement: no such column: ...)
```

which appears to suggest that there is no column but that is because it has a different name than expected. For an example of what happens:

```

> # this only applies to old versions of sqldf and DBI
> # based on example by Adrian Dragulescu
> DF <- data.frame(index=1:12, date=rep(c(Sys.Date()-1, Sys.Date()), 6),
+   group=c("A","B","C"), value=round(rnorm(12),2))
>
> library(sqldf)
> sqldf("select * from DF")
  index date group value
1      1 14259.0    A  -0.24
2      2 14260.0    B   0.16
3      3 14259.0    C   1.24
4      4 14260.0    A  -1.16
5      5 14259.0    B  -0.19
6      6 14260.0    C   0.65
7      7 14259.0    A  -1.24
8      8 14260.0    B  -0.34
9      9 14259.0    C  -0.27
10     10 14260.0    A  -0.18
11     11 14259.0    B   0.57
12     12 14260.0    C  -0.83
> intersect(names(DF), tolower(.SQL92Keywords))
[1] "index" "date" "group" "value"
> DF2 <- DF
> # change column names to i, d, g and v
> names(DF2) <- substr(names(DF), 1, 1)
> sqldf("select * from DF2")
   i      d g      v
1  1 2009-01-16 A  0.35
2  2 2009-01-17 B -0.96
3  3 2009-01-16 C  0.76
4  4 2009-01-17 A  0.07
5  5 2009-01-16 B  0.03
6  6 2009-01-17 C  0.19
7  7 2009-01-16 A -2.03
8  8 2009-01-17 B  0.98
9  9 2009-01-16 C -1.21
10 10 2009-01-17 A -0.67
11 11 2009-01-16 B  2.49
12 12 2009-01-17 C -0.63

```

3. Why does sqldf(“select var(x) from DF”) not work?

The SQL statement passed to sqldf must be a valid SQL statement understood by the database. The functions that are understood include simple SQLite functions and aggregate SQLite functions and functions in the [RSQLite.extfuns](#) package. Thus in this case in place of var(x) one could use variance(x) from the RSQLite.extfuns package. For SQLite functions see the lists of [core functions](#), [aggregate functions](#) and [date and time functions](#).

If each group is not too large we can use group_concat to return all group members and then later use apply in R to use R functions to aggregate results. For example, in the following we summarize the data using sqldf and then apply a function based on var:

```

> DF <- data.frame(a = 1:8, g = gl(2, 4))
> out <- sqldf("select group_concat(a) groupa from DF group by g")
> out
  groupa
1 1,2,3,4
2 5,6,7,8
> out$var <- apply(out, 1, function(x) var(as.numeric(strsplit(x, ",")[1])))
> out
  groupa      var
1 1,2,3,4 1.666667
2 5,6,7,8 1.666667

```

4. How does sqldf work with “Date” class variables?

The H2 database has specific support for Date class variables so with H2 Date class variables work as expected:

```

> library(RH2) # driver support for dates was added in RH2 version 0.1-2
> library(sqldf)

```

```
> test1 <- data.frame(sale_date = as.Date(c("2008-08-01", "2031-01-09",
+ "1990-01-03", "2007-02-03", "1997-01-03", "2004-02-04")))
> as.numeric(test1[[1]])
[1] 14092 22288 7307 13547 9864 12452
> sqldf("select MAX(sale_date) from test1")
      MAX..sale_date..
1      2031-01-09
```

In R, Date class dates are stored internally as the number of days since 1970-01-01 – often referred to as the UNIX Epoch. (They are stored this way on non-UNIX platforms as well.) When the dates are transferred to SQLite they are stored as these numbers in SQLite. (sqldf has a heuristic that attempts to ascertain whether the column represents a Date but if it cannot ascertain this then it returns the numeric internal version.)

In SQLite this is what happens:

The examples below use RSQLite 0.11-0 (prior to that version they would return wrong answers. With RSQLite it will return the correct answer but Date class columns will be returned as numeric if sqldf's heuristic cannot automatically determine if they are to be of class "Date". If you name the output column the same name as an input column which has "Date" class then it will correctly infer that the output is to be of class "Date" as well.

```
> library(sqldf)
> test1 <- data.frame(sale_date = as.Date(c("2008-08-01", "2031-01-09",
+ "1990-01-03", "2007-02-03", "1997-01-03", "2004-02-04")))

> as.numeric(test1[[1]])
[1] 14092 22288 7307 13547 9864 12452

> # correct except that it returns the numeric internal representation
> dd <- sqldf("select max(sale_date) from test1")
> dd
      max(sale_date)
1      22288

> # fix it up
> dd[[1]] <- as.Date(dd[[1]], "1970-01-01")
> dd
      max(sale_date)
1      2031-01-09

> # even better it returns Date class if we name column same as a Date class input column
> sqldf("select max(sale_date) sale_date from test1")
      sale_date
1 2031-01-09
```

Also note this code:

```
> library(sqldf)
> DF <- data.frame(a = Sys.Date() + 1:5, b = 1:5)
> DF
      a b
1 2009-07-31 1
2 2009-08-01 2
3 2009-08-02 3
4 2009-08-03 4
5 2009-08-04 5
> Sys.Date() + 2
[1] "2009-08-01"
> s <- sprintf("select * from DF where a >= %d", Sys.Date() + 2)
> s
[1] "select * from DF where a >= 14457"
> sqldf(s)
      a b
1 2009-08-01 2
2 2009-08-02 3
3 2009-08-03 4
4 2009-08-04 5

> # to compare against character string store a as character
> DF2 <- transform(DF, a = as.character(a))
> sqldf("select * from DF2 where a >= '2009-08-01'")
      a b
1 2009-08-01 2
2 2009-08-02 3
3 2009-08-03 4
4 2009-08-04 5
```



```
1 2009-08-01 2
2 2009-08-02 3
3 2009-08-03 4
4 2009-08-04 5
```

See [date and time functions](#) for more information. An example using times but not dates can be found [here](#) and some discussion on using POSIXct can be found [here](#).

5. I get a message about the tcltk package being missing.

The sqldf package uses the gsubfn package for parsing and the gsubfn package optionally uses the tcltk R package which in turn uses string processing language, tcl, internally.

If you are getting errors about the tcltk R package being missing or about tcl/tk itself being missing then:

Windows. This should not occur on Windows with the standard distributions of R. If it does you likely have a version of R that was built improperly and you will have to get a complete properly built version of R that was built to work with tcltk and tcl/tk and includes tcl/tk itself.

Mac. This should not occur on **recent** versions of R on Mac. If it does occur upgrade your R installation to a recent version. If you must use an older version of R on the Mac then get tcl/tk here: <http://cran.us.r-project.org/bin/macosx/tools/>

UNIX/Linux. If you don't already have tcl/tk itself on your system try this to install it like this (thanks to Eric Iversion):

```
sudo apt-get install tcl-dev tk-dev
```

Also see this message by Rolf Turner: <https://stat.ethz.ch/pipermail/r-help/2011-April/274424.html>.

In some cases it may be possible to bypass the need for tcltk and tcl/tk altogether by running this command before you run sqldf:

```
options(gsubfn.engine = "R")
```

In that case the gsubfn package will use alternate R code instead of tcltk (however, it will be slightly slower).

Notes: sqldf depends on gsubfn for parsing and gsubfn optionally uses the tcltk R package (tcl is a string processing language) which is supposed to be included in every R installation. The tcltk R package relies on tcl/tk itself which is included in all standard distributions of R on Windows on **recent** Mac distributions of R. Many Linux distributions include tcl/tk itself right in the Linux distribution itself.

Also note that whatever build of R you are using must have had tcl/tk present at the time R was built (not just at the time its used) or else the R build process will automatically turn off tcltk capability within R. If that is the case supplying tcltk and tcl/tk later won't help. You must use a build of R that has tcltk capability built in. (If the R was built with tcltk capability then adding the tcltk package (if its missing) and tcl/tk will work.)

6. Why are there problems when we use table names or column names that are the same except for case?

SQL is case insensitive so table names a and A are the same as far as SQLite is concerned. Note that in the example below it did produce a warning that something is wrong although that might not be the case in all situations.

```
> a <- data.frame(x = 1:2)
> A <- data.frame(y = 11:12)
> sqldf("select * from a a1, A a2")
  x x
1 1 1
2 1 1
3 2 2
4 2 2
Warning message:
In value[[3L]](cond) :
  RS-DBI driver: (error in statement: table `A` already exists)
```


7. Why are there messages about MySQL?

sqldf can use several different databases. The database is specified in the `drv=` argument to the `sqldf` function. If `drv=` is not specified then it uses the value of the `"sqldf.driver"` global option to determine which database to use. If that is not specified either then if the RPostgreSQL, RMySQL or RH2 package is loaded (it checks in that order) it uses the associated database and otherwise uses SQLite. Thus if you do not specify the database and you have one of those packages loaded it will think you intended to use that database. If it's likely that you will have one of these packages loaded but you do not want to that package with `sqldf` be sure to set the `sqldf.driver` option, e.g. `options(sqldf.driver = "SQLite")`.

8. Why am I having problems with update?

Although data frames referenced in the SQL statement(s) passed to `sqldf` are automatically imported to SQLite, `sqldf` does not automatically export anything for safety reasons. Thus if you update a table using `sqldf` you must explicitly return it as shown in the examples below.

Note that in the select statement we referred to the table as `main.DF` (`main` is always the name of the sqlite database.) If we had referred to the table as `DF` (without qualifying it as being in `main`) `sqldf` would have fetched `DF` from our R workspace rather than using the updated one in the sqlite database.

```
> DF <- data.frame(a = 1:3, b = c(3, NA, 5))
> sqldf(c("update DF set b = a where b is null", "select * from main.DF"))
  a b
1 1 3
2 2 2
3 3 5
```

One other problem can arise if the data has factors. Here we would normally get the wrong result because we are asking it to add a value to column `b` that is not among the factor levels in `b` but by using `method = "raw"` we can tell it not to automatically assign classes to the result.

```
> DF <- data.frame(a = 1:3, b = factor(c(3, NA, 5))); DF
  a    b
1 1     3
2 2 <NA>
3 3     5
> sqldf(c("update DF set b = a where b is null", "select * from main.DF"), method = "raw")
  a b
1 1 3
2 2 2
3 3 5
```

Another way around this is to avoid the entire problem in the first place by not using a factor for `b`. If we had defined column `b` as character or numeric instead of factor then we would not have had to specify `method = "raw"`.

9. How do I examine the layout that SQLite uses for a table? which tables are in the database? which databases are attached?

Try these approaches to get the indicated meta data:

```
> # a. what is the layout of the BOD table?
> sqldf("pragma table_info(BOD)")
  cid  name type notnull dflt_value pk
1   0   Time REAL      0      <NA>  0
2   1 demand REAL      0      <NA>  0

> # b. which tables are in current database and what is their layout?
> sqldf(c("select * from BOD", "select * from sqlite_master"))
  type name tbl_name rootpage
1 table BOD      BOD        2

                                sql
1 CREATE TABLE `BOD` \n( "Time" REAL,\n\tdemand REAL \n)

> # c. which databases are attached? (This says only 'main' is attached.)
> sqldf("pragma database_list")
```

```

seq name file
1 0 main

> # d. which version of sqlite is being used?
> sqldf("select sqlite_version()")
  sqlite_version()
1          3.7.17

```

10. What are some of the differences between using SQLite and H2 with sqldf?

sqldf will use the H2 database instead of sqlite if the [RH2](#) package is loaded. Features supported by H2 not supported by SQLite include Date class columns and certain [functions](#) such as VAR_SAMP, VAR_POP, STDDEV_SAMP, STDDEV_POP, various XML functions and CSVREAD.

Note that the examples below require RH2 0.1-2.6 or later.

Here are some commands. The meta commands here are specific to H2 (for SQLite's meta data commands see [FAQ#9](#)):

```

library(RH2) # this package contains the H2 database and an R driver
library(sqldf)
sqldf("select avg(demand) mean, stddev_pop(demand) from BOD where Time > 4")
sqldf('select Species, "Sepal.Length" from iris limit 3') # Sepal.Length has dot
sqldf("show databases")
sqldf("show tables")
sqldf("show tables from INFORMATION_SCHEMA")
sqldf("select * from INFORMATION_SCHEMA.settings")
sqldf("select * FROM INFORMATION_SCHEMA.indexes")
sqldf("select VALUE from INFORMATION_SCHEMA.SETTINGS where NAME = 'info.VERSION'")
sqldf("show columns from BOD")
sqldf("select H2VERSION()") # this requires a later version of H2 than comes with RH2

```

If RH2 is loaded then it will use H2 so if you wish to use SQLite anyways then either use the drv= argument to sqldf:

```
sqldf("select * from BOD", drv = "SQLite")
```

or set the following global option:

```
options(sqldf.driver = "SQLite")
```

When using H2:

- in H2 a column such as Sepal.Length is not converted to Sepal_Length (which older versions of RSQLite do) but remains as Sepal.Length. For example,

Also sqlite orders the result above even without the order clause and h2 translates “Sepal Length” to Sepal.Length .

- quoting rules in H2 are stricter than in SQLite. In H2, to quote an identifier use double quotes whereas to quote a constant use single quotes.
- file objects are not supported. They are not really needed because H2 supports a [CSVREAD](#) function. Note that on Windows one can use the R notation ~ to refer to the home directory when specifying filenames if using SQLite but not with CSVREAD in H2.
- currently the only SQL statements supported by sqldf when using H2 are select, show and call (whereas all are supported with SQLite).
- H2 does not support the using clause in SQL select statements but does support on. Also it implicitly uses on rather than using in natural joins which means that selected and where condition variables that are merged in natural joins must be qualified in H2 but need not be in SQLite.

The examples in the Examples section are redone below using H2. Where H2 does not support the operation the SQLite code is given instead. Note that this section is a bit out of date and some of the items that it says are not supported actually are supported now.

```

# 1
sqldf('select * from iris order by "Sepal.Length" desc limit 3')

# 2
sqldf('select Species, avg("Sepal.Length") from iris group by Species')

# 3
sqldf('select iris.Species "[Species]",
      avg("Sepal.Length") "[Avg of SLs > avg SL]"
      from iris,
      (select Species, avg("Sepal.Length") SLavg
       from iris group by Species) SLavg
      where iris.Species = SLavg.Species
      and "Sepal.Length" > SLavg
      group by iris.Species')

# 4
Abbr <- data.frame(Species = levels(iris$Species),
                  Abbr = c("S", "Ve", "Vi"))

# 4a. This works:
sqldf('select iris.Species, count(*)
      from iris natural join Abbr group by iris.Species')

# but this does not work (but does in sqlite) ###
sqldf('select Abbr, count(*)
      from iris natural join Abbr group by Species')

# 4b. H2 does not support using but does support on (but query is longer) ###
sqldf('select Abbr, count(*)
      from iris join Abbr on iris.Species = Abbr.Species group by iris.Species')

# 4c.
sqldf('select Abbr, avg("Sepal.Length") from iris, Abbr
      where iris.Species = Abbr.Species group by iris.Species')

# 4d. # This still needs to be fixed. #
out <- sqldf("select s.Species, s.dt, t.Station_id, t.Value
              from species s, temp t
              where ABS(s.dt - t.dt) =
                (select min(abs(s2.dt - t2.dt))
                 from species s2, temp t2
                 where s.Species = s2.Species and t.Station_id = t2.Station_id)")

# 4e. H2 does not support using but we can use on (but query is longer) ###
# Also the missing value in x seems to get filled with 0 rather than NA ###
SNP1x <- structure(list(Animal = c(194073197L, 194073197L, 194073197L,
194073197L, 194073197L),
                      Marker = structure(1:5,
                                           .Label = c("P1001", "P1002", "P1004", "P1005", "P1006", "P1007"),
                                           class = "factor"),
                      x = c(2L, 1L, 2L, 0L, 2L),
                      .Names = c("Animal", "Marker", "x"),
                      row.names = c("3213", "1295", "915", "2833", "1487"), class = "data.frame")
SNP4 <- structure(list(Animal = c(194073197L, 194073197L, 194073197L,
194073197L, 194073197L, 194073197L),
                      Marker = structure(1:6, .Label = c("P1001",
"P1002", "P1004", "P1005", "P1006", "P1007"), class = "factor"),
                      Y = c(0.021088, 0.021088, 0.021088, 0.021088, 0.021088, 0.021088),
                      .Names = c("Animal", "Marker", "Y"), class = "data.frame",
                      row.names = c("3213", "1295", "915", "2833", "1487", "1885"))

sqldf("select SNP4.Animal, SNP4.Marker, Y, x
      from SNP4 left join SNP1x
      on SNP4.Animal = SNP1x.Animal and SNP4.Marker = SNP1x.Marker")

# 4f. This still needs to be fixed. #

DF <- structure(list(tt = c(3, 6)), .Names = "tt", row.names = c(NA,
-2L), class = "data.frame")
DF2 <- structure(list(tt = c(1, 2, 3, 4, 5, 7), d = c(8.3, 10.3, 19,
16, 15.6, 19.8)), .Names = c("tt", "d"), row.names = c(NA, -6L
), class = "data.frame", reference = "A1.4, p. 270")
out <- sqldf("select * from DF d, DF2 a, DF2 b

```

```

    where a.row_names = b.row_names - 1 and d.tt > a.tt and d.tt <= b.tt",
    row.names = TRUE)

# 5
minSL <- 7
limit <- 3
fn$sqldf('select * from iris where "Sepal.Length" > $minSL limit $limit')

# 6a. Species get converted to upper case ###

#   alternative 1
write.table(head(iris, 3), "iris3.dat", sep = ",", quote = FALSE, row.names = FALSE)

# convert factor to numeric
fac2num <- function(x) UseMethod("fac2num")
fac2num.factor <- function(x) as.numeric(as.character(x))
fac2num.data.frame <- function(x) replace(x, TRUE, lapply(x, fac2num))
fac2num.default <- identity

sqldf("select * from csvread('iris3.dat')", method = function(x)
  data.frame(fac2num(x[-5]), x[5]))

#   alternative 2 (H2 seems to get confused regarding case of Species)
sqldf('select
  cast("Sepal.Length" as real) "Sepal.Length",
  cast("Sepal.Width" as real) "Sepal.Width",
  cast("Petal.Length" as real) "Petal.Length",
  cast("Petal.Width" as real) "Petal.Width",
  SPECIES from csvread(\'iris3.dat\)')

#   alternative 3. 1st line sets up 0 row table, iris0, with correct classes & 2nd line
#   inserts the data from iris3.dat into it and then selects it back.

iris0 <- read.csv("iris3.dat", nrow = 1)[0L, ]
sqldf(c("insert into iris0 (select * from csvread('iris3.dat'))",
  "select * from iris0"))

# 6b.
sqldf("select * from csvread('iris3.dat')", dbname = tempfile(), method = function(x)
  data.frame(fac2num(x[-5]), x[5]))

# 6c. Same answer as in 6a works whether or not there are row names

# 6d. NA

# 6e.

# 6f.
cat("1 8.3
210.3

319.0
416.0
515.6
719.8
", file = "fixed")
sqldf("select substr(V1, 1, 1) f1, substr(V1, 2, 4) f2
  from csvread('fixed', 'V1') limit 3")

# 6g. NA

# 7a

# this is sqlite (how do you work with rowid's in H2?) ###
sqldf('select * from iris i
  where rowid in
    (select rowid from iris where Species = i.Species order by "Sepal.Length" desc limit 2)
  order by i.Species, i."Sepal.Length" desc')

# 7b - same question ###

library(chron)
DF <- data.frame(x = 101:200, tt = as.Date("2000-01-01") + seq(0, len = 100, by = 2))

```

```

DF <- cbind(DF, month.day.year(unclass(DF$tt)))

# sqlite:
sqldf("select * from DF d
      where rowid in
      (select rowid from DF
       where year = d.year and month = d.month and day >= 21 limit 1)
      order by tt")

# 7c.
a <- read.table(textConnection("st en
1 4
11 14
3 4"), header = TRUE)

b <- read.table(textConnection("st en
2 5
3 6
30 44"), TRUE)

sqldf("select * from a where
      (select count(*) from b where a.en >= b.st and b.en >= a.st) > 0")

# 8. In H2 one uses csvread rather than file and file.format. See:
# https://www.h2database.com/html/functions.html#csvread

numStr <- as.character(1:100)
DF <- data.frame(a = c(numStr, "Hello"))
write.table(DF, file = "tmp99.csv", quote = FALSE, sep = ",")
sqldf("select * from csvread('tmp99.csv') limit 5")

# Note that ~ does not work on Windows in H2: ###
# sqldf("select * from csvread('~tmp.csv')")

# 9 - RH2 does not support. Only select statements currently. ###

# create new empty database called mydb
sqldf("attach 'mydb' as new")

# create a new table, mytab, in the new database
# Note that sqldf does not delete tables created from create.
sqldf("create table mytab as select * from BOD", dbname = "mydb")

# shows its still there
sqldf("select * from mytab", dbname = "mydb")

# 10 - RH2 does not support sqldf() ###

sqldf()
# uses connection just created
sqldf('select * from iris3 where "Sepal.Width" > 3')
sqldf('select * from main.iris3 where "Sepal.Width" = 3')
sqldf()

> # Example 10b.
> #
> # Here is another way to do example 10a. We use the same iris3,
> # iris3.dat and sqldf development version as above.
> # We grab connection explicitly, set up the database using sqldf and then
> # for the second call we call dbGetQuery from RSQLite.
> # In that case we don't need to qualify iris3 as main.iris3 since
> # RSQLite would not understand R variables anyways so there is no
> # ambiguity.

> con <- sqldf()
>
> # uses connection just created
> sqldf('select * from iris3 where "Sepal.Width" > 3')
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1         5.1         3.5         1.4         0.2  setosa
2         4.7         3.2         1.3         0.2  setosa
> dbGetQuery(con, 'select * from iris3 where "Sepal.Width" = 3')

```

```

  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1           4.9           3           1.4           0.2  setosa
>
> # close
> sqldf()

# 11. Between - these work same as sqlite

seqdf <- data.frame(thetime=seq(100,225,5),thevalue=factor(letters))
boundsdf <- data.frame(thestart=c(110,160,200),theend=c(130,180,220),groupID=c(555,666,777))

# run the query using two inequalities
testquery_1 <- sqldf("select seqdf.thetime, seqdf.thevalue, boundsdf.groupID
from seqdf left join boundsdf on (seqdf.thetime <= boundsdf.theend) and (seqdf.thetime >=
boundsdf.thestart)")

# run the same query using 'between...and' clause
testquery_2 <- sqldf("select seqdf.thetime, seqdf.thevalue, boundsdf.groupID
from seqdf LEFT JOIN boundsdf ON (seqdf.thetime BETWEEN boundsdf.thestart AND boundsdf.theend)")

# 12 combine two files - not supported by RH2 ###

# 13 see #8

```

11. Why am I having difficulty reading a data file using SQLite and sqldf?

SQLite is fussy about line endings. Note the `eol` argument to `read.csv.sql` can be used to specify line endings if they are different than the normal line endings on your platform. e.g.

```
read.csv.sql("myfile.dat", eol = "\n")
```

`eol` can also be used as a component to the `sqldf` `file.format` argument.

12. How does one use sqldf with PostgreSQL?

Install 1. PostgreSQL, 2. RPostgreSQL R package 3. `sqldf` itself. RPostgreSQL and `sqldf` are ordinary R package installs.

Make sure that you have created an empty database, e.g. "test". The `createdb` program that comes with PostgreSQL can be used for that. e.g. from the console/shell create a database called test like this:

```
createdb --help
createdb --username=postgres test
```

Here is an example using RPostgreSQL and after that we show an example using RpgSQL. The options statement shown below can be entered directly or alternately can be put in your `.Rprofile`. The values shown here are actually the defaults:

```
options(sqldf.RPostgreSQL.user = "postgres",
  sqldf.RPostgreSQL.password = "postgres",
  sqldf.RPostgreSQL.dbname = "test",
  sqldf.RPostgreSQL.host = "localhost",
  sqldf.RPostgreSQL.port = 5432)
```

```
Lines <- "Group_A Group_B Group_C Value
A1 B1 C1 10
A1 B1 C2 20
A1 B1 C3 30
A1 B2 C1 40
A1 B2 C2 10
A1 B2 C3 5
A1 B2 C4 30
A2 B1 C1 40
A2 B1 C2 5
A2 B1 C3 2
A2 B2 C1 26
A2 B2 C2 1
A2 B3 C1 23"
```

```
A2 B3 C2 15
A2 B3 C3 12
A3 B3 C4 23
A3 B3 C5 23"
```

```
DF <- read.table(textConnection(Lines), header = TRUE, as.is = TRUE)

library(RPostgreSQL)
library(sqldf)
# upper case is folded to lower case by default so surround DF with double quotes
sqldf('select count(*) from "DF" ')

sqldf('select *, rank() over (partition by "Group_A", "Group_B" order by "Value")
      from "DF"
      order by "Group_A", "Group_B", "Group_C" ')
```

For another example using over and partition by see: [this cumsum example](#)

Also note that log and log10 in R correspond to ln and log, respectively, in PostgreSQL.

13. How does one deal with quoted fields in read.csv.sql?

read.csv.sql provides an interface to sqlite's csv reader. That reader is not very flexible (but is fast) and, in particular, it does not understand quoted fields but rather regards the quotes as part of the field itself. To read a file using read.csv.sql and remove all double quotes from it at the same time on Windows try this assuming you have Rtools installed and on your path (or the corresponding tr syntax on UNIX depending on your shell):

```
read.csv.sql("myfile.csv", filter = 'tr.exe -d ^" ' )
```

or equivalently:

```
read.csv.sql("myfile.csv", filter = list('gawk -f prog', prog = '{ gsub(/"/, ""); print }') )
```

Another program to look at is the [csvfix](#) program (this is a free external program – not an R program). For example suppose we have commas in two contexts: (1) as separators between fields and within double quoted fields. To handle that case we can use csvfix to translate the separators to semicolon stripping off the double quotes at the same time (assuming we have installed csvfix and we have put it in our path):

```
read.csv.sql("myfile.csv", sep = ";", filter = "csvfix write_dsv -s ;")` .
```

14. How does one read files where numeric NAs are represented as missing empty fields?

Translate the empty fields to some number that will represent NA and then fix it up on the R end.

```
# The problem is that SQLite's read routine regards empty
# fields as zero length character strings rather than NA.
# We handle that by replacing such strings with -999, say,
# using gawk and the read.csv.sql filter argument and then
# fixing it up in R later.

# write out test data

cat("a\tb\tc
aa\t\t23
aaa\t34.6\t
aaaa\t\t77.8", file = "x.txt")

# create single line awk program to insert -999 as NA

cat('{ gsub("\t\t", "\t-999\t"); gsub("\t$", "\t-999"); print}',
    file = "x.awk")

# on Windows gawk uses \n as eol even though most
# other programs use \r\n so we need to specify that.
# eol= may or may not be needed here on other platforms.
```



```
library(sqldf)
DF <- read.csv.sql("x.txt", sep = "\t", eol = "\n", filter = "gawk -f x.awk")

# replace -999's with NA

is.na(DF) <- DF == -999
```

Another program that can be used in filters is the free `csvfix`. For example, suppose that `csvfix` is on our path and that NA values are represented as NA in numeric fields. We would like to convert them to -999 and then later remove them.

```
Lines <- "a,b
3,NA
4,65"
cat(Lines, file = "myfile.csv")

filter <- 'csvfix map -fv ,NA -tv ,-999 myfile.csv | csvfix write_dsv -s ,'
DF <- read.csv.sql(filter = filter)
is.na(DF) <- DF == -999
```

Another way in which the input file can be malformed is that not every line has the same number of fields. In that case `csvfix pad -n` can be used to pad it out as in this example:

```
Lines <- "a,b,c
a,b,
a,b
q,r,t"
cat(Lines, file = "c.csv")
DF <- read.csv.sql(filter = "csvfix pad -n 3 c.csv | csvfix write_dsv -s ,")
```

15. Why do certain calculations come out as integer rather than double?

SQLite/RSQLite, h2/RH2, PostgreSQL all perform integer division on integers; however, RMySQL/MySQL performs real division.

```
> DF <- data.frame(a = 1:2, b = 2:1)
> str(DF) # columns are integer
'data.frame': 2 obs. of 2 variables:
 $ a: int 1 2
 $ b: int 2 1
> #
> # using sqlite - integer division
> sqldf("select a/b as quotient from DF")
  quotient
1         0
2         2
> # force real division
> sqldf("select (a+0.0)/b as quotient from DF")
  quotient
1      0.5
2      2.0
> # force real division
> sqldf("select cast(a as real)/b as quotient from DF")
  quotient
1      0.5
2      2.0
> # insert into table with real columns
> sqldf(c("create table mytab(a real, b real)",
+ "insert into mytab select * from DF",
+ "select a/b as quotient from mytab"))
  quotient
1      0.5
2      2.0
>
> # convert all columns to numeric using method= argument
> # Requires sqldf 0.4-0 or later
>
> tonum <- function(DF) replace(DF, TRUE, lapply(DF, as.numeric))
> sqldf("select a/b as quotient from DF", method = list("auto", tonum))
  quotient
```

```

1      0.5
2      2.0
>
> # use RMySQL - uses real division
> # Requires sqldf 0.4-0 or later
> library(RMySQL)
> sqldf("select a/b as quotient from DF")
  quotient
1      0.5
2      2.0

```

16. How can one read a file off the net or a csv file in a zip file?

Use `read.csv.sql` and specify the URL of the file:

```

# 1
URL <- "https://www.wnba.com/liberty/media/NYL2011ScheduleV3.csv"
DF <- read.csv.sql(URL, eol = "\r")

```

Since files off the net could have any end of line be careful to specify it properly for the file of interest.

As an alternative one could use the `filter` argument. To use this `wget` ([download](#), [Windows](#)) must be present on the system command path.

```

# 2 - same URL as above
DF <- read.csv.sql(eol = "\r", filter = paste("wget -O - ", URL))

```

Here is an example of reading a zip file which contains a single file that is a csv :

```
DF <- read.csv.sql(filter = "7z x -so anscombe.zip 2>NUL")
```

In the line of code above it is assumed that `7z` ([download](#)) is present and on the system command path. The example is for Windows. On UNIX use `/dev/null` in place of `NUL`.

If we had a `.tar.gz` file it could be done like this:

```
DF <- read.csv.sql(filter = "tar xOfz anscombe.tar.gz")
```

assuming that `tar` is available on our path. (Normally `tar` is available on Linux and on Windows its available as part of the [Rtools](#) distribution on CRAN.)

Note that `filter` causes the filtered output to be stored in a temporary file and then read into `sqlite`. It does not actually read the data directly from the net into `sqlite` or directly from the zip or `tar.gz` file to `sqlite`.

Note: The examples in this section assume `sqldf` 0.4-4 or later.

Examples

These examples illustrate usage of both `sqldf` and `SQLite`. For `sqldf` with H2 see [FAQ #10](#). For PostgreSQL see [FAQ#12](#). Also the "`sqldf-unitTests`" demo that comes with `sqldf` works under `sqldf` with `SQLite`, H2, PostgreSQL and MySQL. David L. Reiner has created some further examples [here](#) and Paul Shannon has examples [here](#).

Example 1. Ordering and Limiting

Here is an example of sorting and limiting output from an SQL select statement on the iris data frame that comes with R. Note that although the iris dataset uses the name `Sepal.Length` older versions of the RSQLite driver convert that to `Sepal_Length`; however, newer versions do not. After installing `sqldf` in R, just type the first two lines into the R console (without the `>`):

```

> library(sqldf)
> sqldf('select * from iris order by "Sepal.Length" desc limit 3')

  Sepal.Length Sepal.Width Petal.Length Petal.Width  Species
1           7.9         3.8         6.4         2.0 virginica

```

2	7.7	3.8	6.7	2.2 virginica
3	7.7	2.6	6.9	2.3 virginica

Example 2. Averaging and Grouping

Here is an example which processes an SQL select statement whose functionality is similar to the R aggregate function.

```
> sqldf('select Species, avg("Sepal.Length") from iris group by Species')
```

	Species	avg(Sepal.Length)
1	setosa	5.006
2	versicolor	5.936
3	virginica	6.588

Example 3. Nested Select

Here is a more complex example. For each Species, find the average Sepal Length among those rows where Sepal Length exceeds the average Sepal Length for that Species. Note the use of a subquery and explicit column naming:

```
> sqldf("select iris.Species '[Species]',
+       avg(\"Sepal.Length\") '[Avg of SLs > avg SL]'
+       from iris,
+       (select Species, avg(\"Sepal.Length\") SLavg
+       from iris group by Species) SLavg
+       where iris.Species = SLavg.Species
+       and \"Sepal.Length\" > SLavg
+       group by iris.Species")
```

	[Species]	[Avg of SLs > avg SL]
1	setosa	5.313636
2	versicolor	6.375000
3	virginica	7.159091

```
> # same - using only core R - based on discussion with Dennis Toddenroth
> aggregate(Sepal.Length ~ Species, iris, function(x) mean(x[x > mean(x)]))
  Species Sepal.Length
1   setosa    5.313636
2 versicolor    6.375000
3  virginica    7.159091
```

Note that PostgreSQL is the only free database that supports [window functions](#) (similar to ave function in R) which would allow a different formulation of the above. For more on using sqldf with PostgreSQL see [FAQ #12](#)

```
> library(RPostgreSQL)
> library(sqldf)
> tmp <- sqldf('select
+   "Species",
+   "Sepal.Length",
+   "Sepal.Length" - avg("Sepal.Length") over (partition by "Species") "above.mean"
+   from iris')
> sqldf('select "Species", avg("Sepal.Length")
+   from tmp
+   where "above.mean" > 0
+   group by "Species"')
  Species      avg
1   setosa 5.313636
2  virginica 7.159091
3 versicolor 6.375000
>
> # or, alternately, we could perform the above two steps in a single statement:
>
> sqldf('
+ select "Species", avg("Sepal.Length")
+ from
+   (select "Species",
+    "Sepal.Length",
+    "Sepal.Length" - avg("Sepal.Length") over (partition by "Species") "above.mean"
+   from iris) a
+ where "above.mean" > 0
```

```
+ group by "Species")
  Species      avg
1   setosa 5.313636
2 versicolor 6.375000
3  virginica 7.159091
```

which in R corresponds to this R code (i.e. `partition...over` in PostgreSQL corresponds to `ave` in R):

```
> tmp <- with(iris, Sepal.Length ~ ave(Sepal.Length, iris, FUN = mean))
> aggregate(Sepal.Length ~ Species, subset(tmp, above.mean > 0), mean)
  Species Sepal.Length
1   setosa      5.313636
2 versicolor      6.375000
3  virginica      7.159091
```

Here is some sample data with the correlated subquery from this [Wikipedia page](#):

```
Emp <- data.frame(emp = letters[1:24], salary = 1:24, dept = rep(c("A", "B", "C"), each = 8))

sqldf("SELECT *
FROM Emp AS e1
WHERE salary > (SELECT avg(salary)
FROM Emp
WHERE dept = e1.dept)")
```

Example 4. Join

The different type of joins are pictured in this image: i.imgur.com/1m55Wqo.jpg. (SQLite does not support right joins but the other databases sqldf supports do.) We define a new data frame, `Abbr`, join it with `iris` and perform the aggregation:

```
> # Example 4a.
> Abbr <- data.frame(Species = levels(iris$Species),
+   Abbr = c("S", "Ve", "Vi"))
>
> sqldf('select Abbr, avg("Sepal.Length")
+   from iris natural join Abbr group by Species')

  Abbr avg(Sepal.Length)
1    S           5.006
2   Ve           5.936
3   Vi           6.588
```

Although the above is probably the shortest way to write it in SQL, using `natural join` can be a bit dangerous since one must be very sure one knows precisely which column names are common to both tables. For example, had we included the `row_names` as a column in both tables (by specifying `row.names = TRUE` to `sqldf`) the natural join would not work as intended since the `row_names` columns would participate in the join. An alternate and safer way to write this would be with `join` and `using`:

```
> # Example 4b.
> sqldf('select Abbr, avg("Sepal.Length")
+   from iris join Abbr using(Species) group by Species')

  Abbr avg(Sepal.Length)
1    S           5.006
2   Ve           5.936
3   Vi           6.588
```

or with a `where` clause:

```
> # Example 4c.
> sqldf('select Abbr, avg("Sepal.Length") from iris, Abbr
+   where iris.Species = Abbr.Species group by iris.Species')

  Abbr avg(Sepal.Length)
1    S           5.006
2   Ve           5.936
3   Vi           6.588
```

or a temporal join where the goal is, for each Species/station_id pair, to join the records with the closest date/times.

```
> # Example 4d. Temporal Join
> # see: https://stat.ethz.ch/pipermail/r-help/2009-March/191938.html
>
> library(chron)
>
> Species.Lines <- "Species,Date_Sampled
+ SpeciesB,2008-06-23 13:55:11
+ SpeciesA,2008-06-23 13:43:11
+ SpeciesC,2008-06-23 13:55:11"
>
> species <- read.csv(textConnection(Species.Lines), as.is = TRUE)
> species$dt <- as.numeric(as.chron(species$Date))
>
> Temp.Lines <- "Station_id,Date,Value
+ ANH,2008-06-23 13:00:00,1.96
+ ANH,2008-06-23 14:00:00,2.25
+ BDT,2008-06-23 13:00:00,4.23
+ BDT,2008-06-23 13:15:00,4.11
+ BDT,2008-06-23 13:30:00,4.01
+ BDT,2008-06-23 13:45:00,3.9
+ BDT,2008-06-23 14:00:00,3.82"
>
> temp <- read.csv(textConnection(Temp.Lines), as.is = TRUE)
> temp$dt <- as.numeric(as.chron(temp$Date))
>
> out <- sqldf("select s.Species, s.dt, t.Station_id, t.Value
+ from species s, temp t
+ where abs(s.dt - t.dt) =
+ (select min(abs(s2.dt - t2.dt))
+ from species s2, temp t2
+ where s.Species = s2.Species and t.Station_id = t2.Station_id)")
> out$dt <- chron(out$dt)
> out
  Species          dt Station_id Value
1 SpeciesB (06/23/08 13:55:11)    ANH    2.25
2 SpeciesB (06/23/08 13:55:11)    BDT    3.82
3 SpeciesA (06/23/08 13:43:11)    ANH    2.25
4 SpeciesA (06/23/08 13:43:11)    BDT    3.90
5 SpeciesC (06/23/08 13:55:11)    ANH    2.25
6 SpeciesC (06/23/08 13:55:11)    BDT    3.82
```

A similar but slightly simpler example can be found [here](#).

Here is an example of a left join:

```
> # Example 4e. Left Join
> # https://stat.ethz.ch/pipermail/r-help/2009-April/195882.html
> #
> SNP1x <-
+ structure(list(Animal = c(194073197L, 194073197L, 194073197L,
+ 194073197L, 194073197L), Marker = structure(1:5, .Label = c("P1001",
+ "P1002", "P1004", "P1005", "P1006", "P1007"), class = "factor"),
+ x = c(2L, 1L, 2L, 0L, 2L)), .Names = c("Animal", "Marker",
+ "x"), row.names = c("3213", "1295", "915", "2833", "1487"), class = "data.frame")
>
> SNP4 <-
+ structure(list(Animal = c(194073197L, 194073197L, 194073197L,
+ 194073197L, 194073197L), Marker = structure(1:6, .Label = c("P1001",
+ "P1002", "P1004", "P1005", "P1006", "P1007"), class = "factor"),
+ Y = c(0.021088, 0.021088, 0.021088, 0.021088, 0.021088, 0.021088
+ )), .Names = c("Animal", "Marker", "Y"), class = "data.frame", row.names = c("3213",
+ "1295", "915", "2833", "1487", "1885"))
>
> SNP1x
  Animal Marker x
3213 194073197 P1001 2
1295 194073197 P1002 1
915   194073197 P1004 2
2833 194073197 P1005 0
1487 194073197 P1006 2
> SNP4
```

```

      Animal Marker      Y
3213 194073197 P1001 0.021088
1295 194073197 P1002 0.021088
915   194073197 P1004 0.021088
2833 194073197 P1005 0.021088
1487 194073197 P1006 0.021088
1885 194073197 P1007 0.021088
>
> library(sqldf)
> sqldf("select * from SNP4 left join SNP1x using (Animal, Marker)")
      Animal Marker      Y      x
1  194073197 P1001 0.021088 2
2  194073197 P1002 0.021088 1
3  194073197 P1004 0.021088 2
4  194073197 P1005 0.021088 0
5  194073197 P1006 0.021088 2
6  194073197 P1007 0.021088 NA
> # or if that takes up too much memory
> # create/use/destroy external database
> sqldf("select * from SNP4 left join SNP1x using (Animal, Marker)", dbname = "test.db")
      Animal Marker      Y      x
1  194073197 P1001 0.021088 2
2  194073197 P1002 0.021088 1
3  194073197 P1004 0.021088 2
4  194073197 P1005 0.021088 0
5  194073197 P1006 0.021088 2
6  194073197 P1007 0.021088 NA

> # Example 4f. Another temporal join.
> # join DF2 to row in DF for which DF.tt and DF2.tt are closest
>
> DF <- structure(list(tt = c(3, 6)), .Names = "tt", row.names = c(NA,
+ -2L), class = "data.frame")
> DF
   tt
1   3
2   6
>
> DF2 <- structure(list(tt = c(1, 2, 3, 4, 5, 7), d = c(8.3, 10.3, 19,
+ 16, 15.6, 19.8)), .Names = c("tt", "d"), row.names = c(NA, -6L
+ ), class = "data.frame", reference = "A1.4, p. 270")
> DF2
   tt    d
1   1  8.3
2   2 10.3
3   3 19.0
4   4 16.0
5   5 15.6
6   7 19.8
>
> out <- sqldf("select * from DF d, DF2 a, DF2 b
+ where a.row_names = b.row_names - 1
+ and d.tt > a.tt and d.tt <= b.tt",
+ row.names = TRUE)
>
> out$dd <- with(out, ifelse(tt < (tt.1 + tt.2) / 2, d, d.1))
> out
   tt tt.1    d tt.2  d.1  dd
1   3    2 10.3    3 19.0 19.0
2   6    5 15.6    7 19.8 19.8

```

Example 4g. Self Join. There is an example of a self-join here: [problem](#) and answer here:

```

> DF <- structure(list(Actor = c("Jim", "Bob", "Bob", "Larry", "Alice", "Tom", "Tom", "Tom", "Alice",
+ "Nancy"), Act = c("A", "A", "C",
+ "D", "C", "F", "D", "A", "B", "B")), .Names = c("Actor", "Act"
+ ), class = "data.frame", row.names = c(NA, -10L))

> subset(unique(merge(DF, DF, by = 2)), Actor.x < Actor.y)
   Act Actor.x Actor.y
3    A      Jim      Tom
4    A      Bob      Jim
6    A      Bob      Tom
11   B     Alice    Nancy

```

```
16 C Alice Bob
20 D Larry Tom
```

```
> sqldf("select A.Act, A.Actor, B.Actor
+ from DF A join DF B
+ where A.Act = B.Act and A.Actor < B.Actor
+ order by A.Act, A.Actor")
  Act Actor Actor
1  A   Bob   Jim
2  A   Bob   Tom
3  A   Jim   Tom
4  B Alice Nancy
5  C Alice  Bob
6  D Larry  Tom
```

to Raj Morejoys for correction.

Here is an [another example of a self join](#) to create pairs which is followed by a second self join to produce pairs of pairs. This [stackoverflow example](#) illustrates an sqldf triple join in which one table participates twice.

Example 4h. Join nearby times. There is an example of joining records that are close but not necessarily exactly the same here: [problem](#) and [answer](#) . Also taking successive differences involves joining adjacent times and this is illustrated [here](#) .

Here is an example where we align time series Sy to series Sx by averaging all points of Sy within $w = 0.25$ units of each Sx time point. Tx and X are the times and values of Sx and Ty and Y are the times and values of Sy.

```
Tx <- seq(1, N, 0.5)
Tx <- Tx + rnorm(length(Tx), 0, 0.1)
X <- sin(Tx/10.0) + sin(Tx/5.0) + rnorm(length(Tx), 0, 0.1)
Ty <- seq(1, N, 0.3333)
Ty <- Ty + rnorm(length(Ty), 0, 0.02)
Y <- sin(Ty/10.0) + sin(Ty/5.0) + rnorm(length(Ty), 0, 0.1)
w <- 0.25

system.time(out1 <- sapply(Tx, function(tx) mean(Y[Ty >= tx-w & Ty <= tx+w])))

library(sqldf)
Sx <- data.frame(Tx, X)
Sy <- data.frame(Ty, Y)

system.time(out.sqldf <- sqldf(c("create index idx on Sx(Tx)",
  "select Tx, avg(Y) from main.Sx, Sy
  where Ty + 0.25 >= Tx and Ty - 0.25 <= Tx group by Tx"))))

all.equal(out.sqldf[,2], out1) # TRUE
```

Example 4i. Speeding up joins with indexes. Here is an example of speeding up a join by using indexes on a single join column [here](#) and [here](#) and on two join columns below. Note that the create index statements in each example also has the effect of reading in the data frames into the main database of SQLite. The select statement refers to main.DF1 rather than just DF1 so that it accesses that copy of DF1 in main which we just indexed rather than the unindexed DF1 in R. Similar comments apply to DF2. The statement sqldf("select * from sqlite_master") will list the names and related info for all tables in main.

```
> set.seed(1)
> n <- 1000000
>
> DF1 <- data.frame(a = sample(n, n, replace = TRUE),
+ b = sample(4, n, replace = TRUE), c1 = runif(n))
>
> DF2 <- data.frame(a = sample(n, n, replace = TRUE),
+ b = sample(4, n, replace = TRUE), c2 = runif(n))
>
> library(sqldf)
Loading required package: DBI
Loading required package: RSQLite
Loading required package: gsubfn
Loading required package: proto
Loading required package: chron
>
> sqldf()
```

```

<SQLiteConnection:(6480,0)>
> system.time(sqldf("create index ai1 on DF1(a, b)"))
Loading required package: tcltk
Loading Tcl/Tk interface ... done
  user system elapsed
 16.69   0.19  19.12
> system.time(sqldf("create index ai2 on DF2(a, b)"))
  user system elapsed
 16.60   0.03  17.48
> system.time(sqldf("select * from main.DF1 natural join main.DF2"))
  user system elapsed
   7.76   0.06   8.23
> sqldf()

```

The sqldf statements above could also be done in one sqldf call like this:

```

# define DF1 and DF2 as before
set.seed(1)
n <- 1000000
DF1 <- data.frame(a = sample(n, n, replace = TRUE),
  b = sample(4, n, replace = TRUE), c1 = runif(n))
DF2 <- data.frame(a = sample(n, n, replace = TRUE),
  b = sample(4, n, replace = TRUE), c2 = runif(n))

# combine all sqldf calls from before into one call

result <- sqldf(c("create index ai1 on DF1(a, b)",
  "create index ai2 on DF2(a, b)",
  "select * from main.DF1 natural join main.DF2"))

```

Note that if your data is so large that you need indexes it may be too large to store the database in memory. If you find its overflowing memory then use the dbname= sqldf argument, e.g. sqldf(c("create...", "create...", "select..."), dbname = tempfile()) so that it stores the intermediate results in an external database rather than memory.

Note: The index ai1 is not actually used so we could have saved the time it took to create it, creating only ai2.

```
sqldf(c("create index ai2 on DF2(a, b)", "select * from DF1 natural join main.DF2"))
```

Example 4j. Per Group Max and Min

Note that the Date variable gets passed to SQLite as number of days since 1970-01-01 whereas SQLite uses an earlier origin so we add julianday('1970-01-01') to convert the origin of R's "Date" class to SQLite's origin. Note that the output column called Date is automatically converted to "Date" class by the sqldf heuristic because there is an input column that has the same name.

```

> URL <- "https://ichart.finance.yahoo.com/table.csv?s=G00G&a=07&b=19&c=2004&d=03&e=16&f=2010&g=d&
ignore=.csv"
> DF25 <- read.csv(URL, nrow = 25)
> DF25$Date <- as.Date(DF25$Date)
>
> sqldf("select Date, a.High, a.Low, b.Close, a.Volume
+ from (select max(Date) Date, min(Low) Low, max(High) High, sum(Volume) Volume
+ from DF25
+ group by date(Date + julianday('1970-01-01'), 'start of month')
+ ) as a join DF25 b using(Date)")
      Date      High      Low      Close      Volume
1 2010-03-31  588.28  539.70  567.12  51541600
2 2010-04-16  597.84  549.63  550.15  41201900

```

and here is another shorter one that uses a trick of Magnus Hagander in the second Stackoverflow link below:

```

> sqldf("select
+ max(Date) Date,
+ max(High) High,
+ min(Low) Low,
+ max(100000 * Date + Close) % 100000 Close,
+ sum(Volume) Volume
+ from DF25
+ group by date(Date + julianday('1970-01-01'), 'start of month')")
      Date      High      Low      Close      Volume

```



```
1 2010-03-31 588.28 539.70 567 51541600
2 2010-04-16 597.84 549.63 550 41201900
```

Also see [this Xaprb link](#) for an approach without subqueries and for more discussion see [this stackoverflow link](#) and [this stackoverflow link](#). The last link shows how to use analytical queries which are available in PostgreSQL – the PostgreSQL database, like SQLite and H2, is supported by sqldf.

Example 5. Insert Variables

Here is an example of inserting evaluated variables into a query using [gsubfn](#) quasi-perl-style string interpolation. gsubfn is used by sqldf so its already loaded. Note that we must use the fn\$ prefix to invoke the interpolation functionality:

```
> minSL <- 7
> limit <- 3
> species <- "virginica"
> fn$sqldf("select * from iris where \"Sepal.Length\" > $minSL and species = '$species' limit $limit")
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	7.1	3.0	5.9	2.1	virginica
2	7.6	3.0	6.6	2.1	virginica
3	7.3	2.9	6.3	1.8	virginica

Example 6. File Input

Note that there is a new command `read.csv.sql` which provides an alternate interface to the the approach discussed in this section. See Example 13 for that.

sqldf normally deletes any database it creates after completion but the example sample code [at the bottom of this post](#) shows how to set up a database and read a file into it without having the database destroyed afterwards.

sqldf will not only look for data frames used in the SQL statement but will also look for R objects of class "file". For such objects it will directly import the associated file into the database without going through R allowing files that are larger than an R workspace to be handled and also providing for potential speed advantages. That is, if `f <- file("abc.csv")` is a file object and `f` is used as the table name in the sql statement then the file `abc.csv` is imported into the database as table `f`. With SQLite, the actual reading of the file into the database is done in a C routine in RSQLite so the file is transferred directly to the database without going through R. If the `sqldf` argument `dbname` is used then it specifies a filename (either existing or created by `sqldf` if not existing). That filename is used as a database (rather than memory) allowing larger files than physical memory. By using an appropriate `where` statement or a subset of column names a portion of the table can be retrieved into R even if the file itself is too large for R or for memory.

There are some caveats. The RSQLite `dbWriteTable/sqliteImportFile` routines that `sqldf` uses to transfer the file directly to the database are intended for speed thus they are not as flexible as `read.table`. Also they have slightly different defaults. The default for `sep` is `file.format = list(sep = ",", quote = FALSE)`. If the first row of the file has one fewer component than subsequent ones then it assumes that `file.format = list(header = TRUE, row.names = TRUE)` and otherwise that `file.format = list(header = FALSE, row.names = FALSE)`. `.csv` file format is only partly supported – quotes are not regarded as special.

In addition to the examples below there is an example [here](#) and another one with performance results [here](#).

```
> # Example 6a.
> # test of file connections with sqldf
>
> # create test .csv file of just 3 records
> write.table(head(iris, 3), "iris3.dat", sep = ",", quote = FALSE)
>
> # look at contents of iris3.dat
> readLines("iris3.dat")
[1] "Sepal.Length,Sepal.Width,Petal.Length,Petal.Width,Species"
[2] "1,5.1,3.5,1.4,0.2,setosa"
[3] "2,4.9,3,1.4,0.2,setosa"
[4] "3,4.7,3.2,1.3,0.2,setosa"
>
> # set up file connection
```

```

> iris3 <- file("iris3.dat")
> sqldf('select * from iris3 where "Sepal.Width" > 3')
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1           5.1          3.5          1.4          0.2  setosa
2           4.7          3.2          1.3          0.2  setosa
>
> # Example 6b.
> # similar but uses disk - useful if file were large
> # According to https://www.sqlite.org/whentouse.html
> # SQLite can handle files up to several dozen gigabytes.
> # (Note in this case readTable and readTableIndex in R.utils
> # package or read.table from the base of R, setting the colClasses
> # argument to "NULL" for columns you don't want read in, might be
> # alternatives.)
> sqldf('select * from iris3 where "Sepal.Width" > 3', dbname = tempfile())
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1           5.1          3.5          1.4          0.2  setosa
2           4.7          3.2          1.3          0.2  setosa

> # Example 6c.
> # with this format, header=TRUE needs to be specified
> write.table(head(iris, 3), "iris3a.dat", sep = ",", quote = FALSE,
+ row.names = FALSE)
> iris3a <- file("iris3a.dat")
> sqldf("select * from iris3a", file.format = list(header = TRUE))
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1           5.1          3.5          1.4          0.2  setosa
2           4.9          3.0          1.4          0.2  setosa
3           4.7          3.2          1.3          0.2  setosa

> # Example 6d.
> # header can alternately be specified as object attribute
> attr(iris3a, "file.format") <- list(header = TRUE)
> sqldf("select * from iris3a")
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1           5.1          3.5          1.4          0.2  setosa
2           4.9          3.0          1.4          0.2  setosa
3           4.7          3.2          1.3          0.2  setosa

> # Example 6e.
> # create a test file with all 150 records from iris
> # and select 4 records at random without reading entire file into R
> write.table(iris, "iris150.dat", sep = ",", quote = FALSE)
> iris150 <- file("iris150.dat")
> sqldf("select * from iris150 order by random(*) limit 4")
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1           4.9          2.5          4.5          1.7 virginica
2           4.8          3.0          1.4          0.1  setosa
3           6.1          2.6          5.6          1.4 virginica
4           7.4          2.8          6.1          1.9 virginica
>
> # or use read.csv.sql and its just one line
> read.csv.sql("iris150.dat", sql = "select * from file order by random(*) limit 4")
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1           4.9          2.4          3.3          1.0 versicolor
2           5.8          2.7          4.1          1.0 versicolor
3           7.4          2.8          6.1          1.9 virginica
4           5.1          3.5          1.4          0.3  setosa

```

Example 6f. If our file has fixed width fields rather than delimited then we can still handle it if we parse the lines manually with substr:

```

# write some test data to "fixed"
# Field 1 has width of 1 column and field 2 has 4 columns
cat("1 8.3
210.3
319.0
416.0
515.6
719.8
", file = "fixed")

# get 3 random records using sqldf

```

```
fixed <- file("fixed")
attr(fixed, "file.format") <- list(sep = ";") # ; can be any char not in file
sqldf("select substr(V1, 1, 1) f1, substr(V1, 2, 4) f2 from fixed order by random(*) limit 3")
```

Another example of fixed width data is [here](#) (however, note that changing the sep needs to be done in the example in that link too).

Example 6g. Defaults.

```
# If first row has one fewer columns than subsequent rows then
# header <- row.names <- TRUE is assumed as in example 6a; otherwise,
# header <- row.names <- FALSE is assumed as shown here:

> write.table(head(iris, 3), "iris3nohdr.dat", col.names = FALSE, row.names = FALSE, sep = ",", quote = FALSE)
> readLines("iris3nohdr.dat")
[1] "5.1,3.5,1.4,0.2,setosa" "4.9,3,1.4,0.2,setosa" "4.7,3.2,1.3,0.2,setosa"
> sqldf("select * from iris3nohdr")
  V1 V2 V3 V4 V5
1 5.1 3.5 1.4 0.2 setosa
2 4.9 3.0 1.4 0.2 setosa
3 4.7 3.2 1.3 0.2 setosa
```

Example 7. Nested Select

For each species show the two rows with the largest sepal lengths:

```
> # Example 7a.
> sqldf('select * from iris i
+   where rowid in
+     (select rowid from iris where Species = i.Species order by "Sepal.Length" desc limit 2)
+   order by i.Species, i."Sepal.Length" desc')
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.8	4.0	1.2	0.2	setosa
2	5.7	4.4	1.5	0.4	setosa
3	7.0	3.2	4.7	1.4	versicolor
4	6.9	3.1	4.9	1.5	versicolor
5	7.9	3.8	6.4	2.0	virginica
6	7.7	3.8	6.7	2.2	virginica

Here is a similar example. In this one DF represents a time series whose values are in column x and whose times are dates in column tt. The times have gaps – in fact only every other day is present. The code below displays the first row at or past the 21st of the month for each year/month. First we append year, month and day columns using month.day.year from the chron package and then do the computation using sqldf. (For a version of this using the zoo package rather than sqldf see: <https://stat.ethz.ch/pipermail/r-help/2007-November/145925.html>).

```
> # Example 7b.
> #
> library(chron)
> DF <- data.frame(x = 101:200, tt = as.Date("2000-01-01") + seq(0, len = 100, by = 2))
> DF <- cbind(DF, month.day.year(unclass(DF$tt)))
>
> sqldf("select * from DF d
+   where rowid in
+     (select rowid from DF
+       where year = d.year and month = d.month and day >= 21 limit 1)
+   order by tt")
  x      tt      month      day      year
1 111 2000-01-21         1      21      2000
2 127 2000-02-22         2      22      2000
3 141 2000-03-21         3      21      2000
4 157 2000-04-22         4      22      2000
5 172 2000-05-22         5      22      2000
6 187 2000-06-21         6      21      2000
```

Here is another example of a nested select. We select each row of a for which st/en overlaps with some st/en of b.

```
> # Example 7c.
> #
```

```

> a <- read.table(textConnection("st en
+ 1 4
+ 11 14
+ 3 4"), header = TRUE)
>
> b <- read.table(textConnection("st en
+ 2 5
+ 3 6
+ 30 44"), TRUE)
>
> sqldf("select * from a where
+ (select count(*) from b where a.en >= b.st and b.en >= a.st) > 0")
      st en
1    1   4
2    3   4

```

7d. Another example of a nested select with sqldf is shown [here](#)

Example 8. Specifying File Format

When using file() as used as in Example 6 RSQLite reads in the first 50 lines to determine the column classes. What if they all have numbers in them but then later we start to see letters? In that case we will have to override its choice. Here are two ways:

```

library(sqldf)

# example example 8a - file.format attribute on file.object

numStr <- as.character(1:100)
DF <- data.frame(a = c(numStr, "Hello"))
write.table(DF, file = "~/tmp.csv", quote = FALSE, sep = ",")
ff <- file("~/tmp.csv")

attr(ff, "file.format") <- list(colClasses = c(a = "character"))

tail(sqldf("select * from ff"))

# example 8b - using file.format argument

numStr <- as.character(1:100)
DF <- data.frame(a = c(numStr, "Hello"))
write.table(DF, file = "~/tmp.csv", quote = FALSE, sep = ",")
ff <- file("~/tmp.csv")

tail(sqldf("select * from ff",
  file.format = list(colClasses = c(a = "character"))))

```

Example 9. Working with Databases

sqldf is usually used to operate on data frames but it can be used to store a table in a database and repeatedly query it in subsequent sqldf statements (although in that case you might be better off just using RSQLite or other database directly). There are two ways to do this. In this Example section we show how to do it using the fact that if you specify the database explicitly then it does not delete the database at the end and if you create a table explicitly using create table then it does not delete the table (however, note that that will result in duplicate tables in the database so it will take up twice as much space as one table). A second way to do this is to use persistent connections as shown in the Example section after this one.

```

# create new empty database called mydb
sqldf("attach 'mydb' as new")

# create a new table, mytab, in the new database
# Note that sqldf does not delete tables created from create.
sqldf("create table mytab as select * from BOD", dbname = "mydb")

# shows its still there
sqldf("select * from mytab", dbname = "mydb")

# read a file into the mydb data base using read.csv.sql without deleting it

```

```
#
# 1. First create a test file.
# 2. Then read it into the mydb database we created using the sqldf("attach...") above.
# Since sqldf automatically cleans up after itself we hide
# the table creation in an sql statement so table is not deleted.
# 3. Finally list the table names in the database.

write.table(BOD, file = "~/tmp.csv", quote = FALSE, sep = ",")
read.csv.sql("~/tmp.csv", sql = "create table mytab as select * from file",
  dbname = "mydb")
sqldf("select * from sqlite_master", dbname = "mydb")
```

Example 10. Persistent Connections

These three examples show the use of persistent connections in sqldf. This would be used when one has a large database that one wants to store and then make queries from so that one does not have to reload it on each execution of sqldf. (Note that if one just needs a series of sql statements ending in a single query an alternative would be just to use a vector of sql statements in a single sqldf call.)

```
> # Example 10a.
>
> # create test .csv file of just 3 records (same as example 6)
> write.table(head(iris, 3), "iris3.dat", sep = ",", quote = FALSE)
> # set up file connection
> iris3 <- file("iris3.dat")
> # creates connection so in memory database persists after sqldf call
> sqldf()
<SQLiteConnection:(7384,62)>
>
> # uses connection just created
> sqldf('select * from iris3 where "Sepal.Width" > 3')
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1          3.5          1.4          0.2  setosa
2          4.7          3.2          1.3          0.2  setosa
> # we now have iris3 variable in R workspace and an iris3 table
> # so ensure sqldf uses the one in the main database by writing
> # main.iris3. (Another possibility here would have been to
> # delete the iris3 variable from the R workspace to avoid the
> # ambiguity -- in that case one could just write iris3 instead
> # of main.iris3.)
> sqldf('select * from main.iris3 where "Sepal.Width" = 3')
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          4.9           3          1.4          0.2  setosa
>
> # close
> sqldf()
NULL

> # Example 10b.
> #
> # Here is another way to do example 10a. We use the same iris3,
> # iris3.dat and sqldf development version as above.
> # We grab connection explicitly, set up the database using sqldf and then
> # for the second call we call dbGetQuery from RSQLite.
> # In that case we don't need to qualify iris3 as main.iris3 since
> # RSQLite would not understand R variables anyways so there is no
> # ambiguity.

> con <- sqldf()
>
> # uses connection just created
> sqldf('select * from iris3 where "Sepal.Width" > 3')
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1          3.5          1.4          0.2  setosa
2          4.7          3.2          1.3          0.2  setosa
> dbGetQuery(con, 'select * from iris3 where "Sepal.Width" = 3')
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          4.9           3          1.4          0.2  setosa
>
> # close
> sqldf()
NULL
```

Here is an example of reading a csv file using read.csv.sql and then reading it again using a persistent connection:

```
# Example 10c.

write.table(iris, "iris.csv", sep = ",", quote = FALSE)

sqldf()
read.csv.sql("iris.csv", sql = "select count(*) from file")

# now re-read it from the sqlite database
dd <- sqldf("select * from file")

# now close the connection and destroy the database
sqldf()
```

Example 11. Between and Alternatives

```
# example thanks to Michael Rehberg
#
# build sample dataframes
seqdf <- data.frame(thetime=seq(100,225,5),thevalue=factor(letters))
boundsdf <- data.frame(thestart=c(110,160,200),theend=c(130,180,220),groupID=c(555,666,777))

# run the query using two inequalities
testquery_1 <- sqldf("select seqdf.thetime, seqdf.thevalue, boundsdf.groupID
from seqdf left join boundsdf on (seqdf.thetime <= boundsdf.theend) and (seqdf.thetime >=
boundsdf.thestart)")

# run the same query using 'between...and' clause
testquery_2 <- sqldf("select seqdf.thetime, seqdf.thevalue, boundsdf.groupID
from seqdf LEFT JOIN boundsdf ON (seqdf.thetime BETWEEN boundsdf.thestart AND boundsdf.theend)")
```

Example 12. Combine two files in permanent database

When we issue a series of normal sqldf statements after each one sqldf automatically removes any tables and databases it creates in that statement; however, it does not know about ones that sqlite creates so a database created using attach and the tables created using create table won't be deleted.

Also if sqldf is used without the x= argument (omitting x= denotes the opening of a persistent connection) then objects created in the database including those by sqldf and sqlite are not deleted when the persistent connection is destroyed by the next sqldf statement with no x= argument.

If we have forgotten whether you have a connection open or not we can check either of these:

```
dbListConnections(SQLite()) # from DBI

getOption("sqldf.connection") # set by sqldf
```

Here is an example that illustrates part of the above. See the prior examples for more.

```
> # set up some test data
> write.table(head(iris, 3), "irishead.dat", sep = ",", quote = FALSE)
> write.table(tail(iris, 3), "iristail.dat", sep = ",", quote = FALSE)
>
> library(sqldf)
>
> # create new empty database called mydb
> sqldf("attach 'mydb' as new")
NULL
>
> irishead <- file("irishead.dat")
> iristail <- file("iristail.dat")
>
> # read tables into mydb
> sqldf("select count(*) from irishead", dbname = "mydb")
count(*)
1      3
> sqldf("select count(*) from iristail", dbname = "mydb")
count(*)
```

```

1      3
>
> # get count of all records from union
> sqldf('select count(*) from (select * from main.irishead
+ union
+ select * from main.iris.tail)', dbname = "mydb")
count(*)
1      6

```

Example 13. read.csv.sql and read.csv2.sql

read.csv.sql is an interface to sqldf that works like read.csv in R except that it also provides an sql= argument and not all of the other arguments of read.csv are supported. It uses (1) SQLite's import facility via RSQLite to read the input file into a temporary disk-based SQLite database which is created on the fly. (2) Then it uses the provided SQL statement to read the table so created into R. As the first step imports the data directly into SQLite without going through R it can handle larger files than R itself can handle as long as the SQL statement filters it to a size that R can handle. Here is Example 6c redone using this facility:

```

# Example 13a.
library(sqldf)

write.table(iris, "iris.csv", sep = ",", quote = FALSE, row.names = FALSE)
iris.csv <- read.csv.sql("iris.csv",
  sql = 'select * from file where "Sepal.Length" > 5')

# Example 13b. read.csv2.sql. Commas are decimals and ; is sep.

library(sqldf)
Lines <- "Sepal.Length;Sepal.Width;Petal.Length;Petal.Width;Species
5,1;3,5;1,4;0,2;setosa
4,9;3;1,4;0,2;setosa
4,7;3,2;1,3;0,2;setosa
4,6;3,1;1,5;0,2;setosa
"
cat(Lines, file = "iris2.csv")

iris.csv2 <- read.csv2.sql("iris2.csv", sql = 'select * from file where "Sepal.Length" > 5')

# Example 13c. Use of filter= to process fixed field widths.

# This example assumes gawk is available for use as a filter:
# https://www.icewalkers.com/Linux/Software/514530/Gawk.html
# https://gnuwin32.sourceforge.net/packages/gawk.htm

library(sqldf)
cat("112333
123456", file = "fixed.dat")
cat('BEGIN { FIELDWIDTHS = "2 1 3"; OFS = ","; print "A,B,C" }
{ $1 = $1; print }', file = "fixed.awk")

# the following worked on Windows Vista. One user told me that it only worked if he
# omitted the eol= argument so try it both ways on your system and use the way that
# works for your system.

fixed <- read.csv.sql("fixed.dat", eol = "\n", filter = "gawk -f fixed.awk")

# Example 13d. Read a csv file into the database but do not drop the database or table

# create test file
write.table(iris, "iris.csv", sep = ",", quote = FALSE, row.names = FALSE)

# create an empty database (can skip this step if database already exists)
sqldf("attach mytestdb as new")

# read into table called iris in the mytestdb sqlite database
read.csv.sql("iris.csv", sql = "create table main.iris as select * from file", dbname = "mytestdb")

# look at first three lines
sqldf("select * from main.iris limit 3", dbname = "mytestdb")

# example 13e. Read in only column j of a csv file where j may vary.

```

```
library(sqldf)

# create test data file
nms <- names(anscombe)
write.table(anscombe, "anscombe.dat", sep = ",", quote = FALSE,
            row.names = FALSE)

j <- 2
DF2 <- fn$read.csv.sql("anscombe.dat", sql = "select `nms[j]` from file")
```

Also see this [example](#) and this further [example](#). The latter illustrates the use of the `method=` argument.

Example 14. Use of spatialite library functions

*****This example needs to be revised as automatic loading of spatialite has been removed from sqldf and replaced with the functions in RSQLite.extfuns which are loaded instead*****

This example will only work if spatialite-1.dll is on your PATH. It shows accessing a function in that dll. Other than placing it on your PATH there is no other setup needed. (Note that libspatialite-1.dll is only looked up the first time sqldf runs in a session so you should be sure that it has been put there before starting sqldf.)

```
> library(sqldf)
> # stddev_pop is a function in spatialite library similar to sd in R
> # Note bug: spatialite has stddev_pop and stddev_samp reversed and ditto for var_pop and var_samp. More
on bug at:
> # https://groups.google.com/group/spatialite-users/msg/182f1f629c922607
> sqldf("select avg(demand), stddev_pop(demand) from BOD")
      avg(demand)  stddev_pop(demand)
1    14.833333    4.630623
> c(mean(BOD$demand), sd(BOD$demand))
[1] 14.833333  4.630623
```

Example 15. Use of RSQLite.extfuns library functions

The RSQLite R package includes Liam Healy's extension functions for SQLite. In addition to all the [core functions](#), [date functions](#) and [aggregate functions](#) that SQLite itself provides, the following extension functions are available for use within SQL select statements: **Math:** acos, asin, atan, atan2, acosh, asinh, atanh, difference, degrees, radians, cos, sin, tan, cot, cosh, sinh, tanh, coth, exp, log, log10, power, sign, sqrt, square, ceil, floor, pi. **String:** replicate, charindex, leftstr, rightstr, ltrim, rtrim, trim, replace, reverse, proper, padl, padr, padc, strfilter. **Aggregate:** stdev, variance, mode, median, lower_quartile, upper_quartile. See the bottom of <https://www.sqlite.org/contrib/> for more info on these extension functions.

```
> sqldf("select avg(demand) mean, variance(demand) var from BOD")
      mean      var
1 14.83333 21.44267
> var(BOD$demand)
[1] 21.44267
```

Example 16. Moving Average

This is a simplified version of the example in this [r-help post](#). Here we compute the moving average of x for the 3rd to 9th preceding values of each date performing it separately for each illness.

```
> Lines <- "date      illness x
+ 2006/01/01    DERM 319
+ 2006/01/02    DERM 388
+ 2006/01/03    DERM 336
+ 2006/01/04    DERM 255
+ 2006/01/05    DERM 177
+ 2006/01/06    DERM 377
+ 2006/01/07    DERM 113
+ 2006/01/08    DERM 253
+ 2006/01/09    DERM 316
+ 2006/01/10    DERM 187
+ 2006/01/11    DERM 292
```



```

+ 2006/01/12 DERM 275
+ 2006/01/13 DERM 355
+ 2006/01/01 FEVER 3190
+ 2006/01/02 FEVER 3880
+ 2006/01/03 FEVER 3360
+ 2006/01/04 FEVER 2550
+ 2006/01/05 FEVER 1770
+ 2006/01/06 FEVER 3770
+ 2006/01/07 FEVER 1130
+ 2006/01/08 FEVER 2530
+ 2006/01/09 FEVER 3160
+ 2006/01/10 FEVER 1870
+ 2006/01/11 FEVER 2920
+ 2006/01/12 FEVER 2750
+ 2006/01/13 FEVER 3550"
>
> DF <- read.table(textConnection(Lines), header = TRUE)
> DF$date <- as.Date(DF$date)
>
> sqldf("select
+         t1.date,
+         avg(t2.x) mean,
+         date(min(t2.date) * 24 * 60 * 60, 'unixepoch') fromdate,
+         date(max(t2.date) * 24 * 60 * 60, 'unixepoch') todate,
+         max(t2.illness) illness
+       from DF t1, DF t2
+       where julianday(t1.date) between julianday(t2.date) + 3 and
+ julianday(t2.date) + 9
+         and t1.illness = t2.illness
+       group by t1.illness, t1.date
+       order by t1.illness, t1.date")
  date      mean fromdate      todate illness
1 2006-01-04 319.0000 2006-01-01 2006-01-01 DERM
2 2006-01-05 353.5000 2006-01-01 2006-01-02 DERM
3 2006-01-06 347.6667 2006-01-01 2006-01-03 DERM
4 2006-01-07 324.5000 2006-01-01 2006-01-04 DERM
5 2006-01-08 295.0000 2006-01-01 2006-01-05 DERM
6 2006-01-09 308.6667 2006-01-01 2006-01-06 DERM
7 2006-01-10 280.7143 2006-01-01 2006-01-07 DERM
8 2006-01-11 271.2857 2006-01-02 2006-01-08 DERM
9 2006-01-12 261.0000 2006-01-03 2006-01-09 DERM
10 2006-01-13 239.7143 2006-01-04 2006-01-10 DERM
11 2006-01-04 3190.0000 2006-01-01 2006-01-01 FEVER
12 2006-01-05 3535.0000 2006-01-01 2006-01-02 FEVER
13 2006-01-06 3476.6667 2006-01-01 2006-01-03 FEVER
14 2006-01-07 3245.0000 2006-01-01 2006-01-04 FEVER
15 2006-01-08 2950.0000 2006-01-01 2006-01-05 FEVER
16 2006-01-09 3086.6667 2006-01-01 2006-01-06 FEVER
17 2006-01-10 2807.1429 2006-01-01 2006-01-07 FEVER
18 2006-01-11 2712.8571 2006-01-02 2006-01-08 FEVER
19 2006-01-12 2610.0000 2006-01-03 2006-01-09 FEVER
20 2006-01-13 2397.1429 2006-01-04 2006-01-10 FEVER

```

Because of the date processing this is a bit more conveniently done in H2 with its support of date class. Using the same DF that we just defined. Note that SQL functions like AVG and MIN must be written in upper case when using H2.

```

> library(RH2)
> sqldf("select
+         t1.date,
+         AVG(t2.x) mean,
+         MIN(t2.date) fromdate,
+         MAX(t2.date) todate,
+         t2.illness illness
+       from DF t1, DF t2
+       where t1.date between t2.date + 3 and t2.date + 9
+         and t1.illness = t2.illness
+       group by t1.illness, t1.date
+       order by t1.illness, t1.date")
  date mean fromdate      todate illness
1 2006-01-04 319 2006-01-01 2006-01-01 DERM
2 2006-01-05 353 2006-01-01 2006-01-02 DERM
3 2006-01-06 347 2006-01-01 2006-01-03 DERM

```

4	2006-01-07	324	2006-01-01	2006-01-04	DERM
5	2006-01-08	295	2006-01-01	2006-01-05	DERM
6	2006-01-09	308	2006-01-01	2006-01-06	DERM
7	2006-01-10	280	2006-01-01	2006-01-07	DERM
8	2006-01-11	271	2006-01-02	2006-01-08	DERM
9	2006-01-12	261	2006-01-03	2006-01-09	DERM
10	2006-01-13	239	2006-01-04	2006-01-10	DERM
11	2006-01-04	3190	2006-01-01	2006-01-01	FEVER
12	2006-01-05	3535	2006-01-01	2006-01-02	FEVER
13	2006-01-06	3476	2006-01-01	2006-01-03	FEVER
14	2006-01-07	3245	2006-01-01	2006-01-04	FEVER
15	2006-01-08	2950	2006-01-01	2006-01-05	FEVER
16	2006-01-09	3086	2006-01-01	2006-01-06	FEVER
17	2006-01-10	2807	2006-01-01	2006-01-07	FEVER
18	2006-01-11	2712	2006-01-02	2006-01-08	FEVER
19	2006-01-12	2610	2006-01-03	2006-01-09	FEVER
20	2006-01-13	2397	2006-01-04	2006-01-10	FEVER

Another example which varies somewhat from a strict moving average can be found [in this post](#).

Example 17. Lag

The following example contributed by Søren Højsgaard shows how to lag a column.

```
## Create a lagged variable for grouped data
## -----
# Meaning that in the i'th row we not only have y[i] but also y[i-1].
# This is done on a groupwise basis
library(sqldf)
set.seed(123)
DF <- data.frame(id=rep(1:2, each=5), tvar=rep(1:5,2), y=rnorm(1:10))
# Data with lagged variable added
BB <-
  sqldf("select A.id, A.tvar, A.y, B.y as lag
        from DF as A join DF as B
        where A.rowid-1 = B.rowid and A.id=B.id
        order by A.id, A.tvar")
# Merge with original data:
DD <-
  sqldf("select DF.*, BB.lag
        from DF left join BB
        on DF.id=BB.id and DF.tvar=BB.tvar")
# Do it all in one step:
DD <-
  sqldf("select DF.*, BB.lag
        from DF left join
        (
          select A.id, A.tvar, A.y, B.y as lag
            from DF as A join DF as B
            where A.rowid-1 = B.rowid and A.id=B.id
            order by A.id, A.tvar
        ) as BB
        on DF.id=BB.id and DF.tvar=BB.tvar")
```

In PostgreSQL's [window functions](#) (similar to R's `ave` function) makes reference to other rows particularly easy. Below we repeat the SQLite example in PostgreSQL (except that the following fills with NA):

Be sure PostgreSQL is installed and running.

```
library(RPostgreSQL)
library(sqldf)
sqldf("select *, lag(y) over (partition by id order by tvar) from DF")
```

Example 18. MySQL Schema Information

```
library(RMySQL)
library(sqldf)
sqldf("show databases")
sqldf("show tables")
```

The following SQL statements to query the MySQL table schemas are taken from the [blog of Christophe Ladroue](#):

```
library(RMySQL)
library(sqldf)

# list each schema and its length
sqldf("SELECT TABLE_SCHEMA,SUM(DATA_LENGTH) SCHEMA_LENGTH
      FROM information_schema.TABLES
      WHERE TABLE_SCHEMA!='information_schema'
      GROUP BY TABLE_SCHEMA")

# list each table in each schema and some info about it
sqldf("SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_ROWS, DATA_LENGTH
      FROM information_schema.TABLES
      WHERE TABLE_SCHEMA!='information_schema'")
```

The following SQL statement to query the MySQL table schemas are taken from [the MySQL Performance Blog](#):

```
# Find total number of tables, rows, total data in index size
sqldf("SELECT count(*) tables,
      concat(round(sum(table_rows)/1000000,2),'M') rows,
      concat(round(sum(data_length)/(1024*1024*1024),2),'G') data,
      concat(round(sum(index_length)/(1024*1024*1024),2),'G') idx,
      concat(round(sum(data_length+index_length)/(1024*1024*1024),2),'G') total_size,
      round(sum(index_length)/sum(data_length),2) idxfrac
      FROM information_schema.TABLES")

# find biggest databases
sqldf("SELECT
      count(*) tables,
      table_schema,concat(round(sum(table_rows)/1000000,2),'M') rows,
      concat(round(sum(data_length)/(1024*1024*1024),2),'G') data,
      concat(round(sum(index_length)/(1024*1024*1024),2),'G') idx,
      concat(round(sum(data_length+index_length)/(1024*1024*1024),2),'G') total_size,
      round(sum(index_length)/sum(data_length),2) idxfrac
      FROM information_schema.TABLES
      GROUP BY table_schema
      ORDER BY sum(data_length+index_length) DESC LIMIT 10")

# data distribution by storage engine
sqldf("SELECT engine,
      count(*) tables,
      concat(round(sum(table_rows)/1000000,2),'M') rows,
      concat(round(sum(data_length)/(1024*1024*1024),2),'G') data,
      concat(round(sum(index_length)/(1024*1024*1024),2),'G') idx,
      concat(round(sum(data_length+index_length)/(1024*1024*1024),2),'G') total_size,
      round(sum(index_length)/sum(data_length),2) idxfrac
      FROM information_schema.TABLES
      GROUP BY engine
      ORDER BY sum(data_length+index_length) DESC LIMIT 10")
```

Links

[Visual Representation of SQL Joins](#)